

Thread Priorities, Synchronization, Inter-Thread Communication & Deadlock

1. Thread Priority

A **thread priority** tells Java which thread should get more preference when multiple threads are ready to run.

Java supports priorities from **1 to 10**.

Priority	Meaning
1	Minimum
5	Normal
10	Maximum

Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println(  
            getName() + " Priority: " + getPriority()  
        );  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        MyThread t1 = new MyThread();  
        MyThread t2 = new MyThread();  
  
        t1.setPriority(2);  
        t2.setPriority(8);  
  
        t1.start();  
        t2.start();  
    }  
}
```

Important

```
setPriority(8);
```

sets the priority.

```
getPriority();
```

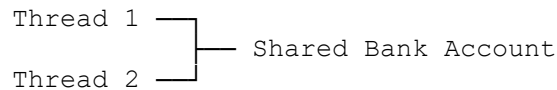
gets the priority.

Higher priority does not guarantee that a thread will execute first. The final scheduling depends on the JVM and operating system.

2. Synchronization

When multiple threads access the **same data**, problems can occur.

Example:



If both threads change the balance simultaneously, the result may be incorrect. This is called a **race condition**.

Synchronization means:

Allowing only one thread at a time to access a critical section.

Think of a **single bathroom**—only one person can use it at a time.

synchronized Method

```
class Counter {  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Counter c = new Counter();  
  
        Thread t1 = new Thread(() -> {  
            for (int i = 0; i < 1000; i++)  
                c.increment();  
        });  
  
        Thread t2 = new Thread(() -> {  
            for (int i = 0; i < 1000; i++)  
                c.increment();  
        });  
  
        t1.start();  
        t2.start();  
  
        t1.join();  
        t2.join();  
    }  
}
```

```
        System.out.println("Count = " + c.count);  
    }  
}
```

Output:

Count = 2000

How it works

Thread 1 → lock → increment() → unlock

↓

Thread 2 → lock → increment() → unlock

synchronized prevents both threads from modifying `count` at the same time.

3. Inter-Thread Communication

Sometimes threads need to **communicate or coordinate** with each other.

Java provides:

```
wait()
notify()
notifyAll()
```

Simple Meaning

Method	Meaning
wait()	Thread waits
notify()	Wakes one waiting thread
notifyAll()	Wakes all waiting threads

These methods are used with synchronization.

Simple Example

```
class Message {

    synchronized void waitForMessage()
        throws InterruptedException {

        System.out.println("Waiting...");
        wait();

        System.out.println("Message received!");
    }

    synchronized void sendMessage() {

        System.out.println("Sending message...");
        notify();
    }
}

public class Main {

    public static void main(String[] args)
        throws InterruptedException {

        Message m = new Message();

        Thread t1 = new Thread(() -> {
```

```

        try {
            m.waitForMessage();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });

    Thread t2 = new Thread(() -> {
        m.sendMessage();
    });

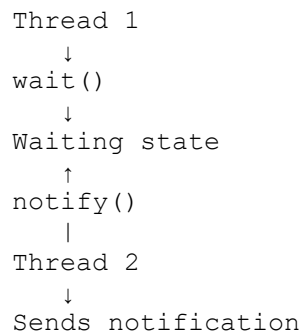
    t1.start();

    Thread.sleep(1000);

    t2.start();
    }
}

```

Flow



`wait()` releases the object's lock while waiting.

4. Producer-Consumer

A common example of inter-thread communication is the **Producer-Consumer problem**.

```
Producer
  ↓
Produces Data
  ↓
Shared Resource
  ↓
Consumer
  ↓
Consumes Data
```

Example:

- Producer → Chef
- Shared resource → Food counter
- Consumer → Waiter

The producer produces something, and the consumer waits if nothing is available.

5. Deadlock

A **deadlock** occurs when two or more threads wait for each other forever.

Real-life example

```
Person A has Pen
Person B has Paper
```

```
A needs Paper
B needs Pen
```

Neither can continue.

Java Example

```
class Resource {  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Resource r1 = new Resource();  
        Resource r2 = new Resource();  
  
        Thread t1 = new Thread(() -> {  
            synchronized (r1) {  
                System.out.println("T1 locked R1");  
  
                synchronized (r2) {  
                    System.out.println("T1 locked R2");  
                }  
            }  
        });  
  
        Thread t2 = new Thread(() -> {  
            synchronized (r2) {  
                System.out.println("T2 locked R2");  
  
                synchronized (r1) {  
                    System.out.println("T2 locked R1");  
                }  
            }  
        });  
  
        t1.start();  
        t2.start();  
    }  
}
```

Possible situation

Thread 1 → Locks R1 → waits for R2
Thread 2 → Locks R2 → waits for R1

↓
DEADLOCK

The program may stop responding because both threads are waiting for each other.

6. How to Avoid Deadlock?

Use locks in the same order.

Bad:

Thread 1 → R1 → R2
Thread 2 → R2 → R1

Better:

Thread 1 → R1 → R2
Thread 2 → R1 → R2

Other practices:

- Avoid unnecessary locks.
- Keep synchronized sections small.
- Release resources as soon as possible.
- Use a consistent locking order.

7. Quick Revision

Topic	Simple Meaning
Thread Priority	Gives preference to a thread
Synchronization	Allows safe access to shared data
Race Condition	Incorrect result due to simultaneous access
synchronized	Provides locking
wait()	Makes a thread wait
notify()	Wakes one waiting thread
notifyAll()	Wakes all waiting threads
Inter-thread Communication	Threads communicate/coordinate
Deadlock	Threads wait for each other forever

Easy Memory Trick

Priority → Who gets preference?
Synchronization → Who can access?
Communication → Who should wait/talk?
Deadlock → Why is everyone stuck?